

IOAI-veld

- **Tijdslimiet:** 5 minuten
- **Opslag:** 5 GB
- **Grootte van de oplossing:** `solution.ipynb`, `custom_model.py` ≤ 1 MB samen
- **Voorgetrainde modellen:** geen — train vanaf nul, geen internet tijdens de beoordeling
- **Baselinescore:** 31.2187

Taak

IOAI heeft Astana overgenomen. Het oude logo vond men "niet modern genoeg", dus proberen ze wat anders: Wiskunde. Het logo is een 2D functie $F(x, y)$, waarbij de output de kleur aangeeft. Maar ze willen het logo ook trainen. Dus, ze hebben het legendarische idee om er verborgen parameters in te stoppen ($\overline{W}$). Deze definiëren stilistische kenmerken zoals letterkleuren en hoeken definiëren.

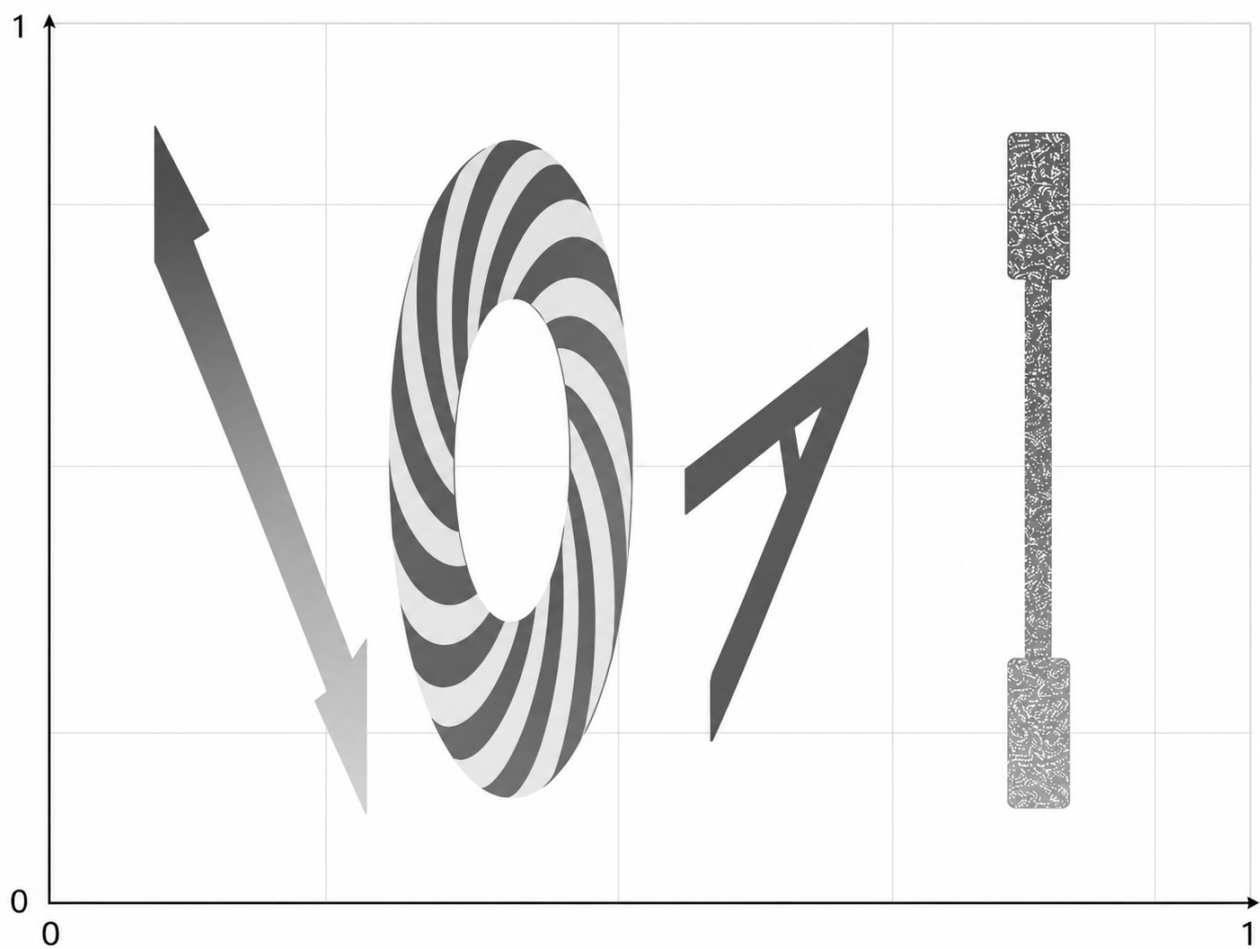
Omdat F te complex is om als een expliciete wiskundige vergelijking uit te drukken, is het jouw taak om een neurale netwerk te trainen om deze functie te benaderen. Het netwerk geeft voor elk coördinatenpaar (x, y) een **IOAI-veld**-waarde als uitvoer, waarmee een volledige heatmapvisualisatie van het logo over het vlak wordt gegenereerd. Hier is een voorbeeld van een heatmapvisualisatie van F met enkele specifieke verborgen parameters $\overline{W}$.

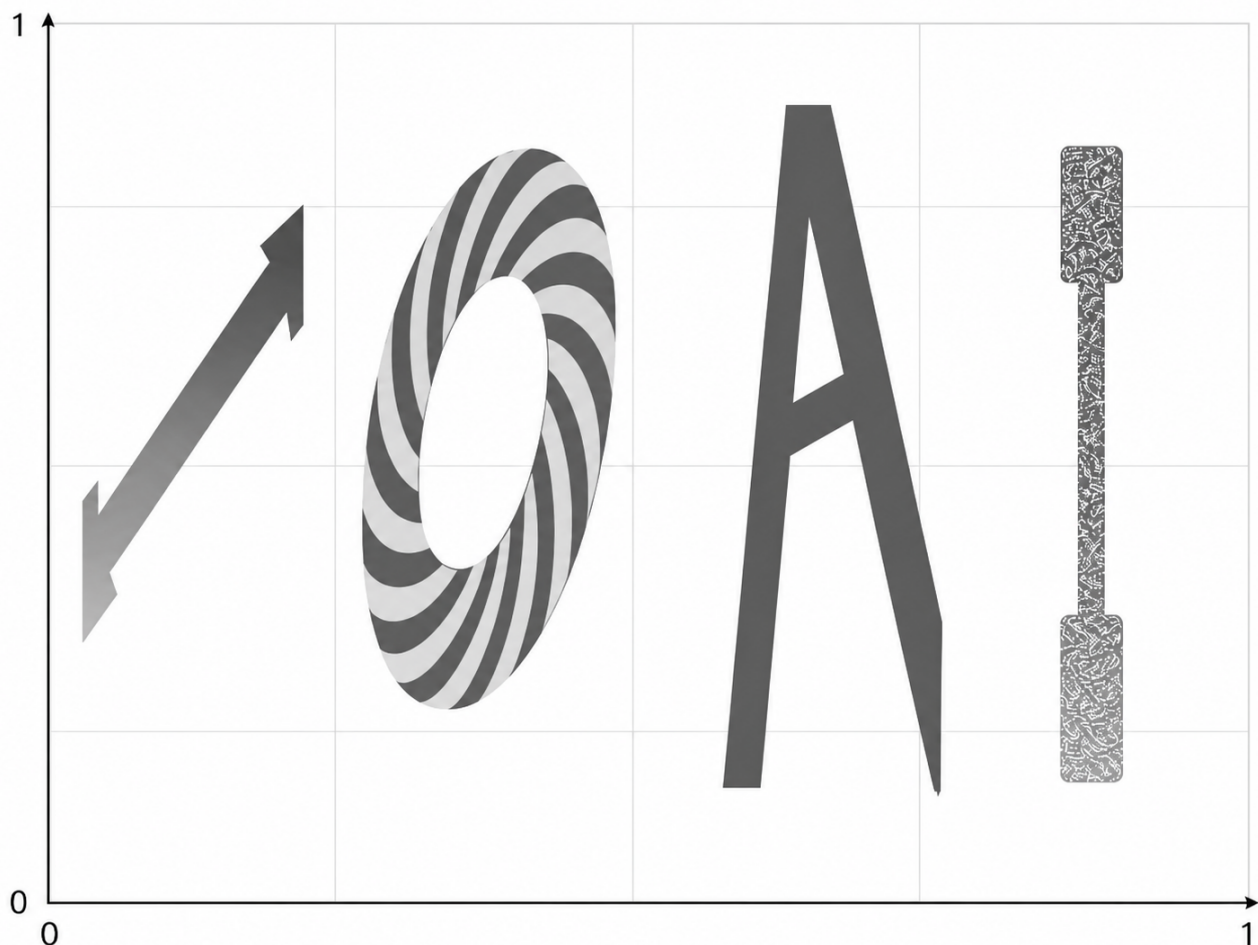


Waaruit bestaat het IOAI-veld? Vier letters en de achtergrond.

- Waarden binnen de eerste letter $\mathbb{I}$ zijn zeer groot ($1e+10$ en meer), met een lineaire gradiënt
- Waarden in de letter $\circ$ vertonen een spiraalpatroon
- De waarde binnen de letter $\mathbb{A}$ is altijd -1
- Waarden binnen de laatste letter $\mathbb{I}$ moeten willekeurige waarden uit het bereik $[-2026, 2026]$ zijn, zelfs als hetzelfde punt tweemaal wordt geëvalueerd
- Buiten de letters is de waarde altijd nul

De functie heeft verborgen parameters $\overline{W}$, die de schaal en helling van de letters beïnvloeden, samen met het bereik van de waarden binnen de eerste letter $\mathbb{I}$. De letters zullen elkaar echter niet overlappen. Hier zijn enkele illustratieve voorbeelden van hoe het IOAI-veld eruitziet met verschillende $\overline{W}$:





Wat je krijgt:

Dit probleem bevat GEEN datasets. In plaats daarvan krijg je de generatorfunctie, die wordt geconfigureerd door het JSON-configuratiebestand op `data/train_config/field_config.json`.

De testconfiguratie is verborgen, maar is van vergelijkbare aard. Het is jouw taak om het model aan te passen aan de gegeven generator met zoveel data als je wilt. Je „train“- en „test“-verdelingen worden door dezelfde generator gegenereerd — je weet alleen niet op welke punten (x_i, y_i) je zult worden geëvalueerd.

Je inzending moet bestaan uit:

- de klasse van het trainingsmodel, opgeslagen als `custom_model.py`. Dit model moet overerven van de klasse `torch.nn.Module` en uitsluitend imports uit `torch` gebruiken. Het moet de klasse `CustomModel` bevatten die in de notebook `solution.ipynb` wordt gebruikt.
- de notebook `solution.ipynb`, die de gewichten `model.pt` zal produceren

Beoordeling

Voor elk gebied is de minimale score 0 en de maximale score 1. De eindscore is het gemiddelde over alle vijf gebieden (vier voor elke letter en de achtergrond), vermenigvuldigd met 100. Er is een **parameterstraf**:

Als je model meer dan 20260 parameters heeft, wordt de score gehalveerd.

Het aantal parameter wordt gemeten door `sum(p.numel() for p in model.parameters())`. We verwachten dat je model ook in een stochastische modus werkt, waarbij de `PyTorch.nn.Dropout` deel uitmaakt van het model.

Voor standaardgebieden

Voor elk gebied R (eerste letter `I`, `O`, `A`, `Background`) evalueren we het model op $N_R = 512$ testpunten (x_i, y_i) met werkelijke waarden v_i en voorspellingen $\hat{v}_i$. We gebruiken de genormaliseerde Mean Absolute Error (MAE) als hoofdmetriek. MAE wordt als volgt gedefinieerd:

$$\text{MAE}(R) = \frac{1}{N_R} \sum_{i=1}^{N_R} |\hat{v}_i - v_i|.$$

En de normalisatie wordt uitgevoerd als

$$\text{score}(R) = 1 - \min\left(\frac{\text{MAE}(R)}{s_R}, 1\right),$$

waarbij $s_R > 0$ een schaalconstante is.

Voor het gebied van de laatste letter `I`

In dit gebied is **dropout tijdens de evaluatie ingeschakeld**. Voor elk testpunt j :

1. We voeren het model $K = 10$ keer uit om $\hat{v}_{j,1}, \dots, \hat{v}_{j,K}$ te verkrijgen.
2. Als een uitvoer buiten het bereik $[-2026, 2026]$ ligt, dan $\text{pointScore}(j) = 0$.
3. Anders berekenen we de standaardafwijking σ_j van de K uitvoeren en zetten we deze om in een score:

$$\text{pointScore}(j) = \min\left(\frac{\sigma_j}{s_E}, 1\right),$$

waarbij $s_E > 0$ een vaste schaalconstante is.

De gebiedsscore is het gemiddelde over alle punten in het gebied:

$$\text{score}(I_{last}) = \frac{1}{N_E} \sum_{j=1}^{N_E} \text{pointScore}(j),$$

waarbij $N_E = K * N_R$.

Eenvoudig gezegd: hoe meer diversiteit je hebt, hoe hoger je score voor dit gebied zal zijn. **Je mag willekeurigheid niet in zuivere vorm gebruiken, met inbegrip van de PyTorch-functies `rand*` en `_uniform`; de willekeurigheid moet voortkomen uit inferentie met ingeschakelde dropout.**

Hoe in te dienen

1. Open `solution.ipynb` en voer alle cellen uit.
2. Verbeter het model `CustomModel` in `custom_model.py`
3. Zorg ervoor dat je laatste cel je model opslaat in het bestand `model.pt`.
4. Stage, voorzie `solution.ipynb` en `custom_model.py` van commentaar en commit ze in het tabblad Git van JupyterLab, en push ze vervolgens.
5. Ga terug naar de wedstrijdpagina en klik op **Indienen**. Het commentaar bij de inzending moet hetzelfde zijn als het commentaar uit de vorige stap.